Chapter-04

TF-IDF

Find TF-IDF for d1:

d1 → any big cat

d2 → big cat

d3 → cat dog cat

$$tf(w,d) = \log(1 + f(w,d))$$

$$idf(w,D) = \log\left(\frac{N}{f(w,D)}\right)$$

Ans:

N = 3

vocabulary = {any, big, cat, dog}

for d1,

$tf(any, d1) = \log_e(1+1) = 0.693$

$tf(big, d1) = \log_e(1+1) = 0.693$ (↑ একবার এসেছে)

$tf(cat, d1) = \log_e(1+1) = 0.693$

$tf(dog, d1) = \log_e(1+0) = 0$

$idf(any, D) = \log_e\left(\frac{3}{1}\right) = 1.098$ (↑ কয়টার doc এ ০i Specific word আছে)

$idf(big, D) = \log_e\left(\frac{3}{2}\right) = 0.405$

$idf(cat, D) = \log_e\left(\frac{3}{3}\right) = 0$

$idf(dog, D) = \log_e\left(\frac{3}{1}\right) = 1.098$

$tfidf(any, d_1, D) = 0.693 \times 1.098 = 0.76$

$tfidf(big, d, D) = 0.693 \times 0.405 = 0.281$

$tfidf(cat, d_1, D) = 0.693 \times 0 = 0$

$tfidf(dog, d_1, D) = 0 \times 1.098 = 0$

[0.76, 0.281, 0, 0] → vector (↑ tf idf)

[1, 1, 1, 0] → BOW আসল

Self-study

Syllabus: Slide 4, Word Representation (1-42 pg)

problem with BoW

1. Too sparse
2. Completely ignores word order
3. Almost no semantic information preserved

problem with sparsity:

1. No overlap → wrong similarity

Two sentences can mean the same thing but share no exact words.

'Dogs chew snacks' vs 'Canines eat treats'

2. High dimensional + inefficient

if |V| = 50,000, a BoW vector for one sentence length is 50,000, mostly zeros

if bigrams | trigram added vocab will explode.

3. Harder to generalize

TF-IDF

Term Frequency (TF) = $\log(1 + f(w,d))$ → document

$\frac{\text{No. of rep of words in sentence}}{\text{No. of words in that sentence}}$

Inverse Document Frequency (IDF) = $\log_e \left(\frac{\text{(N) No. of sentences}}{\text{No. of sentences containing the word}} \right)$

Advantages

1. Fixed size → I/P ⇒ Vocab size
2. Word Importance is getting captured

DisAdvantages:

1. Sparsity, 2. out of vocabulary.

$tf(w,d) = \log_e(1 + f(w,d))$

↑ vocabulary র পুরোটার word এর নির্দিষ্ট document/sentence র

→ ঐ word টা ঐ নির্দিষ্ট doc/sentence এ কয়বার repeat হচ্ছে

TF = how much this word appears in this document

DF = in how many documents does this word appear?

IDF = uniqueness/rarity score

High idf = the word is rare across the document, so when it appears, it's a strong clue about the topic.

Low idf = common in everywhere, so it doesn't help identify what the document is about.

Q. Find tf-idf

d_1 = any big cat

d_2 = big cat

d_3 = cat dog cat

Equations:

$$tf(w,d) = \log_e(1 + f(w,d))$$

$$idf(w,D) = \log_e\left(\frac{N}{f(w,D)}\right)$$

N = no. of sentences

f(w,D) → word টা কতটা doc/sentence এ আছে

$N = 3$, vocabulary = {any, big, cat, dog}

Tf:

$tf(any, d_1) = \log_e(1+1) = 0.693$

$tf(big, d_1) = \log_e(1+1) = 0.693$

$tf(cat, d_1) = \log_e(1+1) = 0.693$

$tf(dog, d_1) = \log_e(1+0) = 0$

#high Tf vector is informative, its frequent in the sentences

IDF:

$idf(any, D) = \log_e(\frac{3}{1}) = 1.099$

$idf(big, D) = \log_e(3/2) = 0.405$

$idf(cat, D) = \log_e(3/3) = 0$

$idf(dog, D) = \log_e(3/1) = 1.099$

Tf-idf:

$tf\text{-}idf(any, d_1, D) = 0.693 \times 1.099 = 0.762$

$tf\text{-}idf(big, d_1, D) = 0.693 \times 0.405 = 0.281$

$tf\text{-}idf(cat, d_1, D) = 0.693 \times 0 = 0$

$tf\text{-}idf(dog, d_1, D) = 0 \times 1.099 = 0$

tf-idf representation for d1 = [0.762, 0.281, 0, 0]

Perfect Word Representation

1. shared lemmas (same base word)
   → mouse/mice, dormir/duermes etc.
2. different word senses:
   → computer mouse vs pet mouse
3. synonyms → couch/sofa, car/automobile
4. antonyms → long/short, dark/light
5. word similarity → dog/cat, doctor/nurse (same category/type)
6. Word relatedness (connected but not similiar)
   → cup/coffee, scalpel/surgeon
7. word valence (positive vs negetive emotion)
   → excited, relax ↑, depressed, angry ↓
8. word arousal (calm vs intense)
   → excited, angry ↑
   relaxed, depressed ↓

Term-Term Matrix

V → vocabulary size

$X = |V| \times |V|$

X_{ij} → records how often word j occured in the context of word i

X_i → each row X_i → vector representation for word i

Building a term term matrix:

docs = ["any big cat", "big cat", "cat dog cat"]

- within ±2 word of each other

d_1 = any big cat

d_2 = big cat

d_3 = cat dog cat

loop

	j=0	j=1	j=2	j=3	j=4
i=0		any	big	cat	dog
i=1	any	$X_{any,any}$	$X_{any,big}$ 1	1	
i=2	big	1		2	
i=3	cat	1	2		
i=4	dog				

Easy way to build

- build a binary BoW for the sentences
- Transpose it
- Multiply it with the original matrix

docs = ["any big cat", "big cat", "cat dog cat"]

M =

term	any	big	cat	dog
doc1	1	1	1	0
doc2	0	1	1	0
doc3	0	0	1	1

M =

1	1	1	0
0	1	1	0
0	0	1	1

3X4

M^T =

1	0	0
1	1	0
1	1	1
0	0	1

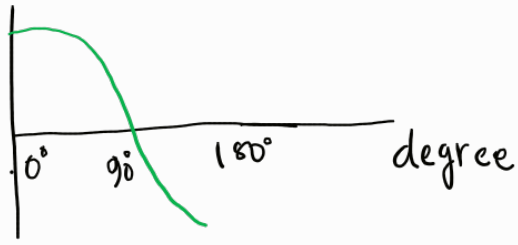
4X3

$M^T . M$ =

	any	big	cat	dog
any	1	1	1	0
big	1	2	2	0
cat	1	2	3	1
dog	0	0	1	1

diagonal values of this matrix represents how many document had that specific word.

Cosine Similarity



cosine graph

$\cos 0^\circ = 1$

$\cos 90^\circ = 0$

$\cos 180^\circ = -1$

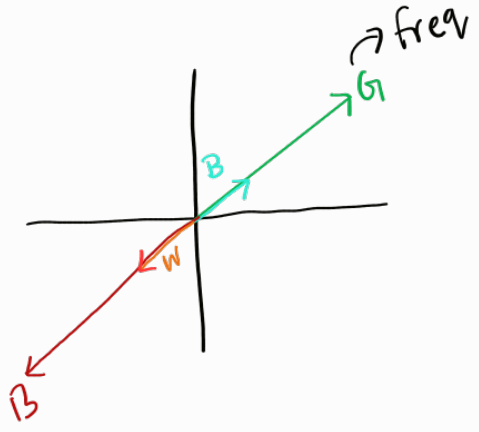
$\therefore$ range is between 1 & -1

feature vector: A numeric vector that describes an item (a word, document, image etc.) in a way a model can use.

$$\text{cosine}(v, w) = \frac{v^T \cdot w}{|v||w|} = \frac{\sum v_i w_i}{\underbrace{\sqrt{\sum v_i^2}}_{\text{length of } v} \underbrace{\sqrt{\sum w_i^2}}_{\text{length of } w}}$$

Why not euclidean?

→



অনেক euclidean distance এ Best এর worst এর distance এর চেয়ে best & good এর distance বেশি হয়ে যায়, which shouldn't be.

we usually care about same direction / same pattern more than same size. Euclidean distance gets dominated by magnitude, cosine focuses on direction.

an example:

$$V = [1,0,1,0]$$

$$W = [3,0,3,0]$$

$$\text{cosine}(V,W) = \frac{1\times3 + 1\times3}{\sqrt{1^2+0^2+1^2+0^2} \times \sqrt{3^2+0^2+3^2+0^2}} =$$

What's wrong with term-term matrix?

Term-term matrix এ আমরা কোন neighboring word কতবার আসছে।

$X_{ij} \Rightarrow$ how often word j appears near word i (co-occurence)

① sparse. huge memory waste, slow computations.

② Doesn't carry any contextual information. It doesn't know meaning changes by context.

③ Doesn't show word importance in a sentence

→ soln

word vectors/embeddings are dense, we compare words using cosine similarity and use embeddings as features. for sentences, we combine word vectors or use RNN to produce a single vector for classification.

Lecture: 4.2

Word2Vec: Skipgram

Date: 10/03/2026

Word embeddings

neighbouring words predict করার try করবো

Continious bag of words → CBOW → human যেহেতু চিন্তা easy

↓

predict the target word given the neighboring words

Skipgram → predict the neighboring words given the target word

↓

it works better

___ ___ red ___ ___

Skipgram Embedding

context window = 2

I live in Dhaka.....

input word	target word
I	live
I	in
live	I
live	in
live	Dhaka

Training an Embedding model

Step 1: for each word in the dataset, create a one hot vector (shape $1 \times |V|$)

Step 2: pass it to the Neural Network to calculate the prediction

Step 3: project to output using softmax

NN Architecture:

Step 1

I live in Dhaka

vocabulary: { I (0), live (1), in (2), Dhaka (3) }

I এর one hot vector,

I = [1, 0, 0, 0]

live = [0, 1, 0, 0]

in = [0, 0, 1, 0]

Dhaka = [0, 0, 0, 1]

[এখন আমরা nn এ দাঁড়াবো। suppose আমরা live এর vector দাঁড়ালে, prediction এ আমাকে in এর vector দাঁড়াতে হবে]

আমাদের training data হল = (live, in)

step 2

live

[0 1 0 0]

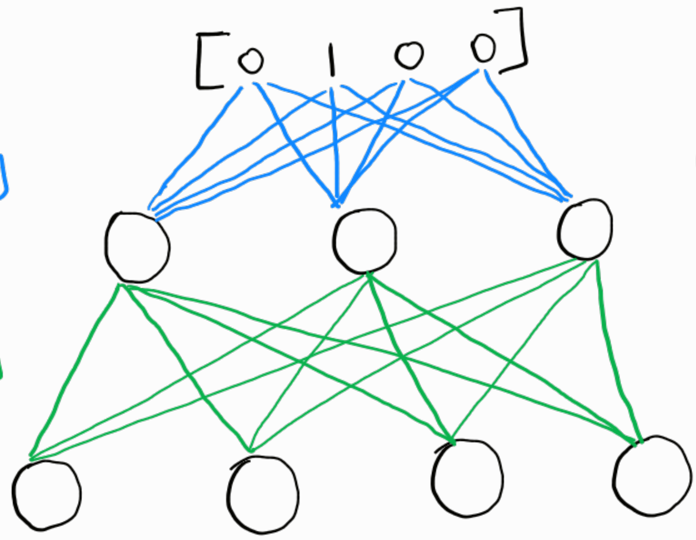
এই nn এ একটাই hidden layer থাকবে।

hidden layer, 3 neuron
↳ K=3

|V| x K
→ Embedding matrix [4x3]

→ context matrix [3X4]
K x |V|

softmax activation function →



0.3 0.1 0.5 0.1
(assuming)

input vector shape [1 x V]

weight matrix = [V x 3] = [4x3]

[1 x 3]

weight matrix = [3X4]

← output layer

এখানে in এর vector predict হওয়া উচিত
[0 0 1 0]

So 4টা neuron output layer এ থাকা লাগবে

model এর prediction

[0.3, 0.1, 0.5, 0.1]

এখন, actual → in = [0, 0, 1, 0]

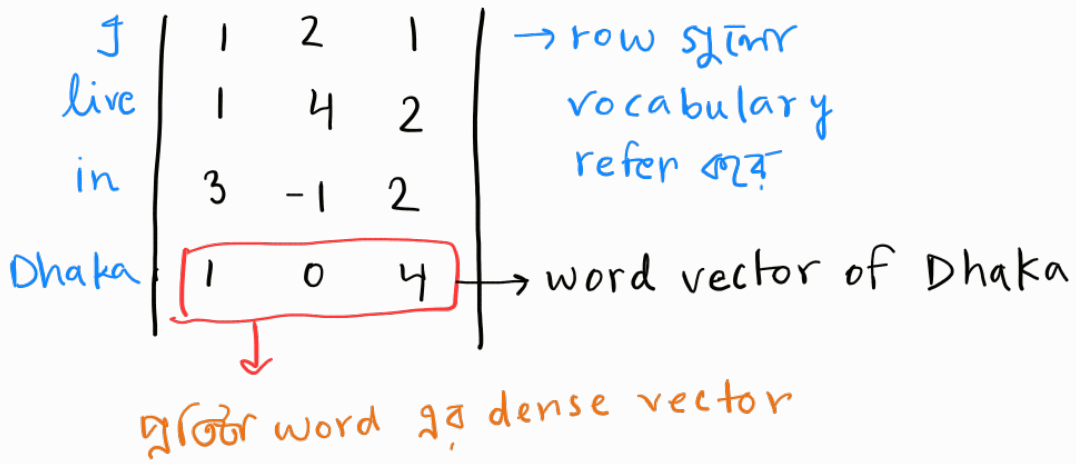
so, error = actual − prediction

এই error গুলোর use করে weight গুলোর update করবো।

Training সময়,

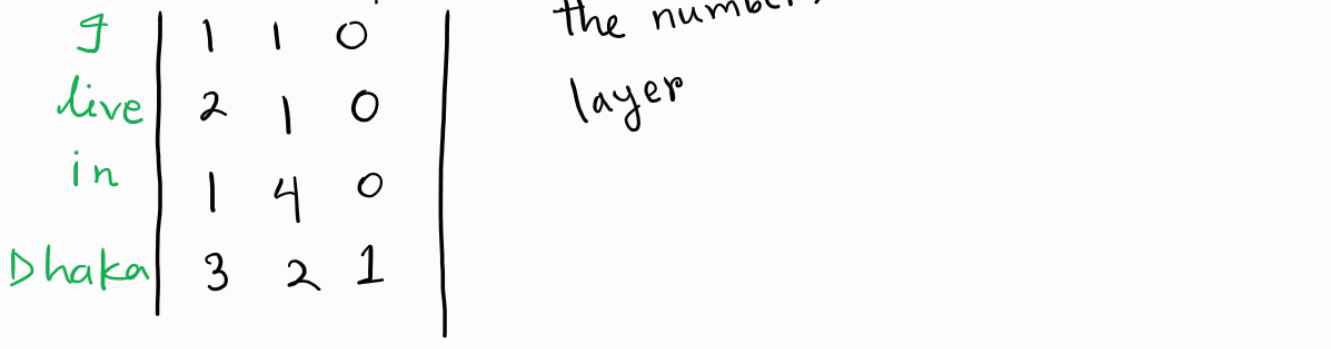
Embedding mat. = |V| x K (→ number of neurons)

= 4x3



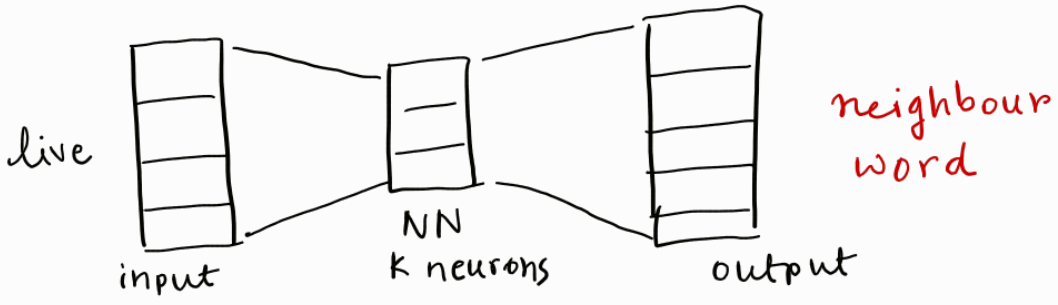
Context matrix = $K \times |V|$

$CM^T = |V| \times K$



by default, skipgram Embedding matrix এর vector গুলোকে word vector হিসেবে consider করে। but another alternative → Em + Cm

skipgram এর আগে আমরা sparse vector use করেছি। but dense vector নেওয়ার জন্য আমরা skipgram use করলাম।



problem

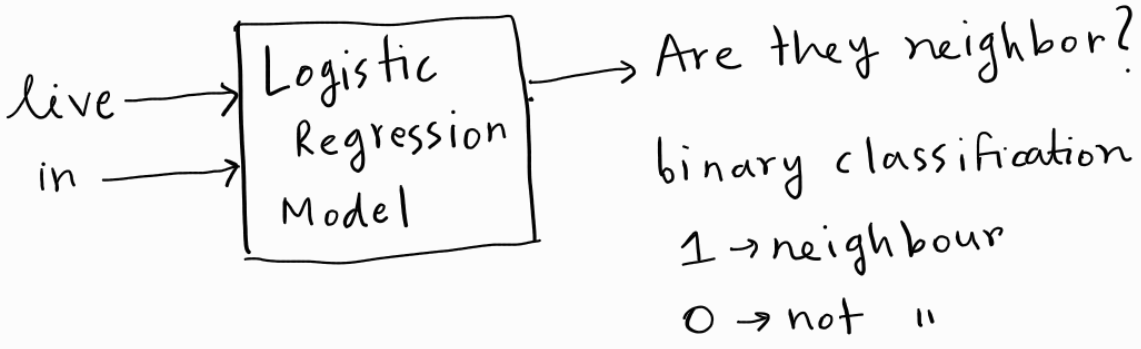
1. step3 computationally very much expensive

assuming, just 1 ta training এ vocab = 50k to 100k

এরকম অনেক training data, and then softmax

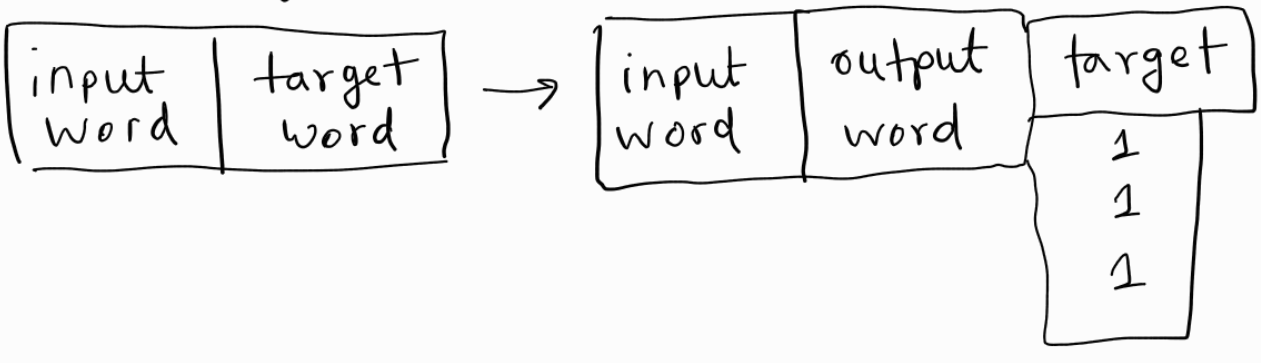
activation function computationally very expensive

To solve → Mikolov's entry



steps

① converting data for the task



② সব 1 এরকম data imbalance হয়ে যাবে

that's why negetive sampling

③

EM
|V| x K

	j	
	1	2	1
live	0	1	1
in	2	4	4
Dhaka	1	1	0

CM^T
|V| x K

	j	
	1	0	-1
live	2	1	0
in	1	0	1
Dhaka	4	1	0

(live, in, 1)

live = [0 1 1] in = [1 0 1]

dot product → similiar value গুলাকে high value পাবো

$(0 \times 1) + (1 \times 0) + (1 \times 1) = 1$

↓ sigmoid

$$\sigma(1) = \frac{1}{1 + e^{-(1)}} = 0.999$$

Then,

Error = Actual − Prediction

= 1 − 0.999 = 0.001

update

update weight

Then gradient calculate